

HARNESS.md — CCAT Question Generator Harness

Purpose

This harness governs a CCAT-style test generation agent. The worker agent's job is narrow: generate one CCAT-style practice question at a time, including answer choices, the correct answer, and an explanation. The harness owns the surrounding control system: iteration, tool access, quality checks, failure handling, persistence, and observability.

The goal is not just to produce questions, but to produce questions that are answerable, correctly solved, appropriately difficult, and traceable from first draft through final acceptance.

Agent Main Loop

The harness runs a controlled generation loop until the requested number of accepted questions has been produced, such as 40 total questions.

For each requested question:

1. Select target category and subcategory, such as Math & Logic, Verbal, or Spatial Reasoning.
2. Ask the worker agent to generate a single question with:
 3. prompt and/or stimulus
 4. answer choices
 5. intended correct answer
 6. explanation
 7. metadata, including category, subcategory, difficulty, and required skills
8. Run the generated question through declared guardrails and checkpoints.
9. If the question fails, return structured feedback to the worker agent and request a revision.
10. Persist every attempt, checkpoint result, revision, and final accepted question.
11. Stop when the accepted question count reaches the requested total or when escalation criteria are met.

The agent is not allowed to silently skip failed checks. Failed checks must either trigger revision, rejection, or human review.

Pillar 1: Main Loop / Checkpoints

The main loop is implemented as a harness-level controller, not inside the worker agent. The worker generates candidates; the harness decides whether they pass.

Each candidate question passes through explicit checkpoints:

Checkpoint	Pass Criteria	Failure Behavior
Schema Check	Required fields are present and valid	Reject and ask agent to regenerate using schema feedback
Answerability Check	Question contains enough information to solve	Ask agent to revise missing or ambiguous information
Single Correct Answer Check	Exactly one answer choice is correct	Ask agent to revise choices or explanation
Explanation Check	Explanation supports the declared correct answer	Ask agent to correct reasoning
Difficulty Check	Difficulty roughly matches requested target	Revise or relabel difficulty
Duplicate / Near-Duplicate Check	Question is not too similar to prior generated or seed questions	Reject and regenerate
Category Fit Check	Question matches requested CCAT category/subcategory	Revise or regenerate
Math Verification Check	Calculations agree with deterministic tool output	Revise calculation, answer choices, or explanation

Checkpoint results are persisted so a run can be replayed from a failed or accepted checkpoint without regenerating earlier material.

Pillar 2: Tools / Material Handling

The harness provides clean interfaces for material flowing into and out of the agent.

Inputs may include:

- requested number of questions
- category distribution
- subcategory taxonomy
- example CCAT-style questions
- difficulty target
- formatting rules
- existing accepted questions to avoid duplication

The worker agent may use harness-approved tools, including:

- exact arithmetic calculator
- symbolic or deterministic math checker
- table generator for data interpretation questions
- graph generator for math or logic questions
- similarity checker against seed and accepted questions

- structured JSON validator

The agent does not call arbitrary tools directly. It requests a tool action through the harness, and the harness records the request, tool input, tool output, and whether the result changed the candidate question.

Generated artifacts are stored as structured question records, not just free text. A final accepted question should be exportable to JSON, CSV, or database rows for use in a study application.

Pillar 3: Guardrails

Guardrails are declared rules that constrain what the worker agent may produce.

Declared guardrails include:

- No copyrighted or copied questions from the seed corpus.
- No unanswerable, ambiguous, or opinion-based questions.
- No questions with multiple valid correct answers.
- No explanations that contradict the selected answer.
- No hidden assumptions required to solve the question.
- No malformed answer choices.
- No math questions whose answer depends on unchecked mental arithmetic.
- No misleading difficulty labels.
- No unsafe, discriminatory, or inappropriate content.
- No final acceptance unless all required checkpoints pass.

When a guardrail fails, the harness returns specific feedback to the worker agent. Example:

```
{
  "guardrail": "single_correct_answer",
  "status": "failed",
  "severity": "high",
  "message": "Choices B and D are both valid under the current wording.",
  "recommended_action": "Revise the stimulus or answer choices so exactly one
option is correct."
}
```

The worker agent's behavior must change based on this feedback. The next revision should directly address the failed guardrail rather than generating an unrelated question.

Pillar 4: Observability / Alarms

The harness records both product-quality events and operational metrics.

Observable quality data:

- original candidate question
- every revision attempt
- checkpoint results
- guardrail failures
- tool calls and outputs
- final accepted version
- reason for rejection or escalation

Observable operational metrics:

- latency per generation
- latency per checkpoint
- total run time
- token usage
- estimated cost
- number of revisions per accepted question
- acceptance rate
- failure rate by checkpoint type
- tool usage frequency

Alarms produce structured output with named alarm type, context, severity, and recommended action.

Example alarms:

```
{
  "alarm_type": "MAX_REVISIONS_EXCEEDED",
  "severity": "medium",
  "context": {
    "question_id": "candidate_017",
    "revision_count": 4,
    "last_failed_checkpoint": "single_correct_answer"
  },
  "recommended_action": "Escalate to human review or discard candidate and regenerate."
}
```

```
{
  "alarm_type": "COST_THRESHOLD_APPROACHING",
  "severity": "high",
  "context": {
    "estimated_cost_usd": 4.75,
    "configured_limit_usd": 5.00
  },
}
```

```
"recommended_action": "Pause run and request confirmation before continuing."
}
```

```
{
  "alarm_type": "LOW_ACCEPTANCE_RATE",
  "severity": "medium",
  "context": {
    "accepted_questions": 8,
    "total_candidates": 30,
    "acceptance_rate": 0.27
  },
  "recommended_action": "Review prompt, taxonomy targeting, or checkpoint
strictness."
}
```

Human-in-the-Loop Escalation

The harness should stop and ask for human review when it cannot safely continue. Escalation triggers include:

- same checkpoint fails more than a configured number of times
- candidate exceeds maximum revision count
- tool output conflicts with agent explanation
- generated question appears too close to a source example
- cost or latency exceeds configured limit
- category taxonomy is unclear
- the harness cannot determine whether the question is valid

This prevents the agent from guessing when the correct behavior is to pause.

Swappable Worker Interface

The harness should treat the worker agent as replaceable. The worker only needs to implement a simple interface:

```
generateQuestion(input: GenerationRequest): CandidateQuestion
reviseQuestion(candidate: CandidateQuestion, feedback: CheckpointFeedback[]):
CandidateQuestion
```

A different worker model or prompt strategy can be swapped in without changing the harness controller, checkpoint system, observability layer, or persistence model.

Demo Plan

At demo time, the harness will run on a real input from the CCAT study project: a requested batch of CCAT-style questions using the project's existing taxonomy and example corpus.

The demo should show:

1. A question being generated.
2. A checkpoint or guardrail failure.
3. The worker revising based on structured feedback.
4. The final accepted question.
5. The persisted trace of all iterations.
6. Observability metrics such as latency, token usage, cost, and revision count.
7. At least one structured alarm or escalation path.

The key design claim is that the agent focuses on generation, while the harness governs constraints, validation, iteration, and visibility.